CodeVedha An Auditable Programming Assessment Platform with Correctness Bounded Scoring

Sri Tulasi Aeloori[1], A. Manigreeva[2], D. Nayaneesh[3], Kantam Pavan Sai Reddy[4], Kura Vignesh Reddy[5]*, Vitta Karthikeya[6]

1 CSE (AI & ML); 2–6 Department of Computer Science and Engineering
Keshav Memorial Institute of Technology
3-5-1026, Narayanguda, Hyderabad, Telangana 500029, India

1 sritulasi447@gmail.com; 2 a.manigreeva@gmail.com

3 nayaneesh6607@gmail.com; 4 kantampavansaireddy30@gmail.com

5 kvigneshreddy963@gmail.com; 6 vittakarthikeya@gmail.com

*Corresponding author: Kura Vignesh Reddy, kvigneshreddy963@gmail.com

Abstract—Programming assessments can reward an efficient program even when it returns wrong answers, while AI feedback can promise gains that have never been tested. CodeVedha addresses both problems by separating observed correctness, inferred complexity, and advice. It combines isolated code execution with a versioned rubric whose final mark cannot exceed the percentage of evaluated cases passed. Instructors can change component weights without removing this constraint. An optional projection route executes a private candidate implementing the displayed guidance and scores it under the saved rubric. On 84 synthetic configurations per rule, weighted aggregation exceeded observed correctness in 49 cases and a fixed failure cap did so in 43; multiplication and the proposed ceiling had no unrounded violations. All eight live execution contracts met their expected outcomes, and candidate scores matched fresh assessments under two rubrics. A diagnostic set of 20 synthetic snippets exposed a failure to recognize equal loop dimensions. These findings support implementation consistency, not expert grading accuracy, optimal weights, or improved student learning.

Index Terms—programming assessment, algorithmic complexity, automated feedback, correctness ceiling, verified score projection, reproducibility

I. Introduction

Automated execution provides a practical foundation for programming assessment, but a passed test suite reveals only behavior on the inputs that were evaluated. A resource estimate describes a different property: how work or storage changes with input size. Readability findings and generated advice introduce further judgments. A useful report should explain how these forms of evidence contribute to the grade and which conclusions remain uncertain. Earlier reviews of test-based assessment and automated programming feedback motivate examining these concerns together [1], [2].

Setting correctness to 100% would enforce a correctness-only grade, but it would give equally correct solutions the same mark regardless of their time, storage, or source quality. A complexity-focused course needs to distinguish those solutions. We therefore separate a mandatory correctness bound from the instructor's allocation of quality credit. The bound applies to every valid weight configuration, including zero internal correctness weight; changing weights through the supported interface cannot remove it. Efficiency can reduce the grade of a correct solution, but cannot lift an incorrect solution above its observed correctness. This is a policy distinction, not a claim that the minimum operator is a new mathematical result.

CodeVedha addresses these problems through explicit evidence boundaries. Students run public examples before submission; the server independently checks the submitted source before evaluating the full test suite. The report stores the execution result, complexity evidence, component scores, and rubric snapshot. Optional language model agents provide explanations and staged guidance, while correctness and score aggregation remain governed by explicit code. Assignment access and due dates are enforced separately from independent practice.

The contribution is an implemented evidence contract linking three decisions: a correctness ceiling on the mark, explicit incompleteness when weighted evidence is missing, and a separately measured candidate projection tied to the saved rubric. We investigate whether these decisions prevent violations of the stipulated ceiling, how they differ from alternative correctness constraints, and whether candidate scoring agrees with a fresh official assessment. The contribution is this integration and its controlled characterization, rather than a new asymptotic inference algorithm. The arithmetic invariant alone does not establish pedagogical validity.

II. Related Work

Douce, Livingstone, and Orwell review automatic test-based programming assessment [1]. Keuning, Jeuring, and Heeren examine automated feedback generation for programming exercises [2]. These studies motivate distinguishing a verdict from the explanation that helps a learner revise a program. Piech et al. investigate learned program representations for propagating instructor feedback [3], illustrating a different route to scaling feedback from examples.

Autograder+ combines adapted language models, instructional prompt selection, and learned representations for feedback and cohort visualization [4]. Its evaluation emphasizes feedback alignment and submission representations. CodeVedha instead studies the contract connecting execution evidence, withheld scores, rubric snapshots, and revision projections. We use Autograder+ as motivation, without inheriting its data, training results, or

numerical feedback claims. The present controlled comparison varies scoring rules within CodeVedha; it is not a head-to-head evaluation of the two platforms.

APPS evaluates generated programs through programming challenges and test execution [5]. HumanEval similarly evaluates functional behavior of generated code [6]. These benchmarks inform the need to execute candidate programs instead of treating plausible source text as sufficient evidence. The present study is smaller and serves a different purpose: it tests implementation contracts and explicitly reports a synthetic, AI-assisted label-agreement pilot. Benchmark pass rates, human feedback agreement, and our regression counts are not interchangeable metrics.

SPEED derives symbolic execution bounds using instrumentation and invariants [7]. Multivariate amortized resource analysis relates several input dimensions through a type-based framework with a soundness argument [8]. CodeVedha instead uses structural heuristics and optional model review; it does not claim comparable soundness or replace these inference methods. Its contribution concerns how uncertain estimates, execution evidence, and candidate scores enter an assessment.

Jonsson and Svingby distinguish scoring reliability from validity and educational consequences [9]. Their review motivates explicit criteria and independent validation, not our exact numerical weights. Thus formal resource analysis addresses bound derivation, programming-feedback systems address learner advice, and CodeVedha studies the consistency of the grading and projection contract connecting these outputs. This positioning is conceptual; no comparative superiority experiment was conducted.

III. Platform Architecture and Evidence Flow

A. Assessment Workflow

Figure 1 separates official assessment from optional projection. A React frontend communicates with a FastAPI backend and PostgreSQL storage; a separate container service executes Python, C, C++, Java, and JavaScript. The report persists execution results, complexity evidence, component scores, and the rubric used for that submission. Specialized workflow steps do not themselves establish superiority over a single model.

Students must pass public examples before submission. The server rechecks eligibility against the exact submitted source, so a browser flag or an earlier successful run cannot authorize changed code. Full assessment evaluates configured public and private cases; passing all determines acceptance. Service outages are distinguished from student compilation or execution failures. Enrollment, instructor policy, and due dates are checked before course execution.

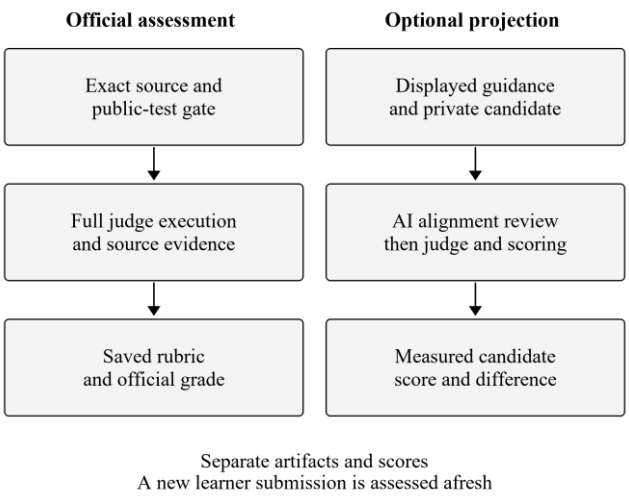


Fig. 1. Separate official assessment and optional candidate projection routes. AI alignment review is distinct from execution verification.

B. Input Contracts and Output Validation

Human-readable labels and nested arrays must be serialized into a documented executable input format. Our nested-list fixtures use an outer count followed by each row's width and values. Thus four interval pairs arrive as count four and four width-two rows. A parser expecting only successive pairs consumes those widths as data. Serialization and parser agreement are prerequisites to judging the algorithm.

The service compiles the requested source in a disposable workspace and supplies each case through standard input. Results retain compilation status, exit behavior, stdout, stderr, and resource-limit outcomes. Comparison normalizes line endings and trailing whitespace, accepts structurally equal JSON, and otherwise compares whitespace-delimited tokens. It does not generally sort output or apply numeric tolerances; some tasks need dedicated comparators.

Runtime containers restrict user privileges, networking, filesystem writes, processes, CPU, and memory using Docker controls [10]. These precautions are not a formal security guarantee. Compilation and container overhead are not algorithmic complexity evidence.

C. Complexity and Style Evidence

The analyzer uses Tree-sitter syntax [11], loop bounds, nesting, recognized operations, and allocation patterns. It records expressions, heuristic confidence, method, status, and supporting signals. Complete structural results at the default direct threshold of 0.85 can proceed without optional model review. This confidence is not a calibrated probability of correctness. For weaker evidence, observational runtime growth and enabled AI review may contribute; sparse timings do not establish asymptotic bounds. Conflicting AI estimates do not silently replace existing structural expressions. Private target complexities stay in backend scoring, and missing estimates do not receive invented full marks.

Time and space percentages compare inferred expressions with curated target ranks on ten ordered buckets: constant, logarithmic, square-root, linear, linearithmic, quadratic, cubic, fourth-degree-or-higher polynomial, exponential, and factorial. Multivariate expressions remain visible, although scoring projects them onto this finite ladder. This is a grading

convention, not universal equivalence of complexity functions; incorrect targets or dimension assumptions can bias marks.

For submitted rank r_s, target rank r_t, and L buckets, the penalty is $25 \max(0, r_s - r_t) / \max(1, L - r_t)$. Subtracting it from 25, clipping to [0, 25], and rounding to three decimals gives component marks; multiplying by four gives the percentage. Both resource components follow this rule. Unmappable expressions require review.

Storage conventions depend on the interface: a supplied function input is excluded from auxiliary space, whereas an input-sized container allocated by a submitted standard-input program counts under the platform policy. Deterministic style checks cover naming, readability, modularity, and maintainability, each contributing up to 25 percentage points. Comments are removed before checking, so missing comments alone do not reduce style. These findings are heuristics, not expert aesthetic judgments.

IV. Versioned Scoring Policy

A. Components and Weights

For a valid assessment with N evaluated cases and p accepted cases, observed correctness is defined by (1). Compilation failure yields zero correctness; an unavailable judge or missing usable execution evidence leaves correctness unknown. Passing a finite suite is not a proof for all valid inputs. Case counts weight each evaluated case equally, so test diversity and duplicates affect the interpretation of partial credit.

$$C = 100\,p / N, \quad N > 0. \quad (1)$$

Table I gives the default rubric. T, M, and Y denote time, space, and style percentages, respectively; C is observed correctness. The weighted quality score and unrounded final policy are given by (2). Instructors may specify nonnegative question or assignment-context weights totaling one hundred percent. The correctness ceiling remains mandatory. Selected weights and the policy version are stored with the submission and cannot silently change when a question is edited later.

TABLE I. DEFAULT VERSIONED RUBRIC

Component	Weight	Evidence
Correctness	30%	Observed case verdicts
Time	35%	Inferred cost and target rank
Space	25%	Allocation evidence and target rank
Style	10%	Deterministic source findings

$$Q = 0.30C + 0.35T + 0.25M + 0.10Y, \quad (2)$$
$$S_0 = min(Q, C).$$

The allocation gives time and space a combined sixty percent of the weighted score, matching the platform's complexity-focused purpose. Correctness retains thirty percent within that sum and independently limits the outcome. Style receives ten percent so source presentation cannot dominate demonstrated behavior. These are transparent design choices, not values proven optimal by a literature citation. A defensible empirical justification would compare alternatives against independently collected instructor judgments on a held-out set. Such grades were not available for this study.

B. Correctness Ceiling and Rounding

Proposition 1: for finite component percentages between zero and one hundred and nonnegative weights summing to one, the unrounded policy satisfies zero less than or equal to S_0 less than or equal to C less than or equal to one hundred. Proof: the weighted sum Q is nonnegative and bounded by one hundred. Taking its minimum with C preserves nonnegativity and makes the result no larger than C. In particular, zero correctness forces zero marks regardless of the other components. This argument applies to every valid instructor weight configuration.

Ordinary decimal rounding could violate the exact ceiling near one hundred. The implemented display mark therefore uses (3), where the round operator denotes the implementation's nearest-tenth rounding. Taking the smaller value prevents a partial correctness fraction from rounding up past its bound. Regression cases include near-perfect fractions rather than only integer percentages.

$$S = min(round_{0.1}(S_0), \lfloor 10C \rfloor / 10). \quad (3)$$

For example, if forty-four of sixty-one cases pass and all remaining components equal one hundred, Q is approximately 91.64, whereas C is approximately 72.13; the displayed score is 72.1. A fixed seventy-nine-point failure cap is unnecessary. A submission can still have a high partial score when very few cases fail, but its verdict remains not accepted, and failed results are not labelled Excellent. Acceptance and numeric quality are separate outputs.

For fixed time, space, and style scores, increasing correctness cannot decrease the unrounded policy: both C and its weighted contribution are nondecreasing. Improving another component cannot increase the mark while correctness is the active ceiling. The report should explain this plateau rather than promise a gain for every optimization. This property constrains arithmetic behavior; it does not validate the underlying complexity estimates or the pedagogical fairness of the weights.

C. Incomplete Evidence

Unknown components with positive weight remain unknown. The platform reports an incomplete assessment and bounds derived from known contributions and remaining possible contributions; it does not renormalize the other weights. Correctness is required even when an instructor assigns it zero weight, because the ceiling still depends on it. A confirmed zero correctness value already determines a zero final mark. An uncertainty-aware report is preferable to presenting a precise total assembled from unsupported assumptions.

V. Evidence Guided Feedback and Candidate Projection

Feedback prioritizes compilation and observed correctness before efficiency or style. The report can show a failed public case or intentionally reveal one failed private case as a learning example; remaining private details are redacted. Repeated submissions can disclose additional cases, a deployment tradeoff rather than a secrecy guarantee. Guided

stages ask the learner to implement one scoped repair or improvement; instructor policy may restrict assistance.

Projection is a separate, optional measurement. A private candidate generator receives source, public context, stage, and exact displayed guidance. An AI-assisted review checks whether the advice is supported, the candidate implements its material actions, and the stage scope is preserved. This is model judgment, not a proof of equivalence. Table II summarizes the eligibility gates and observable outcomes.

TABLE II. CANDIDATE PROJECTION DECISIONS

Condition	Outcome
Missing, unsupported, or misaligned guidance/candidate	Withhold measured projection
Alignment passes; execution or assessment incomplete	Checking or unavailable; no promised gain
All cases pass; complete evidence; saved rubric matches	Measure candidate score and difference
Measured difference is zero or negative	Keep verified no increase
Learner submits a later implementation	Execute and assess it afresh

Only an eligible candidate that passes every configured case and has complete weighted evidence receives a measured score under the original saved rubric. Equation (4) gives the displayed difference. The number belongs to that candidate; it does not guarantee a gain for the learner's later implementation, which is assessed afresh.

$$\Delta = S(candidate) - S(saved\ submission). \quad (4)$$

Zero or negative differences are retained rather than replaced with positive gains. Saved status distinguishes checking, slower verification, paused refresh, and unavailable results. Bounded retries and request identifiers prevent stale work replacing a newer stage. Candidate source and private metadata stay server-side; matching includes source, rubric, and test context. No real-provider latency or success-rate claim follows from the controlled tests.

VI. Verification Method and Results

A. Frozen Implementation and Research Questions

The baseline implementation snapshot is Git commit d4cdb86, with rubric version complexity-focused-v2 and guidance version six, verified on 30 October 2026. Its source and prompt manifest records SHA-256 values, allowed configuration, Python 3.11.16, Linux x86-64, and twelve reported logical processors. Credentials and database contents are excluded. The evidence separates three questions: whether execution and scoring contracts hold; whether static complexity labels agree with supplied AI proposals; and which findings remain insufficient for human grading or educational claims.

The baseline backend suite passed 333 tests, projection rendering passed nine checks, and frontend lint and build passed. Coverage includes arithmetic, incomplete evidence, privacy, stale requests, alignment rejection, and API behavior. Optional provider responses are mocked where needed. These counts establish regression behavior, not grading accuracy.

B. Live Execution and Score Contracts

Eight live execution checks used original synthetic programs and three inputs per check. Table III summarizes the matrix. Sum programs passed positive, negative, and zero inputs in all five languages. C++ Merge Intervals passed endpoint-sharing, unsorted overlapping, and nested-interval fixtures under the documented row-width contract. A changed incorrect source failed all three cases, and restoring the correct source passed again. Thus all eight checks met their expected outcomes; the intentionally incorrect program is not counted as a successful solution.

TABLE III. LIVE SYNTHETIC EXECUTION CONTRACTS

Check	Runs	Passed cases per run
Sum in five languages	5	3/3 each
Nested-array merge in C++	1	3/3
Changed incorrect source	1	0/3 as expected
Restored correct source	1	3/3
Contracts meeting expectation	8/8	No model accuracy claim

A separate contract used an inefficient summation program and a streaming candidate with an isolated in-memory database and the optional provider disabled. Under default weights, the original scored 85.4 and the candidate scored 97.4. Under a custom twenty/fifty/twenty-five/five allocation, the original scored 81.9 and the candidate scored 98.7. In both configurations, the projected candidate score exactly matched a fresh official assessment of the same candidate, and all three candidate cases passed. The differences were 12.0 and 16.8 points, respectively. This checks shared assessment plumbing under two policies; it does not estimate AI generation reliability or validate semantic guidance alignment on real model outputs.

The scoring audit additionally evaluated twenty-eight synthetic component combinations under the default, equal-weight, and correctness-heavier policies. All eighty-four policy-scenario evaluations obeyed the correctness ceiling. Because that ceiling is shared by the comparators, the result confirms arithmetic consistency but does not identify the best allocation. Regression tests and the proposition provide different evidence: examples exercise the implementation, while the argument establishes the stated policy invariant under its assumptions.

C. Diagnostic Complexity Case Study

We use 20 original AI-assisted synthetic function snippets across Python, C, C++, Java, and JavaScript: direct summation, repeated summation, linear search, and binary search. There are only two problem groups, not 20 independent problems. Search inputs are sorted; the convention is worst-case time and auxiliary space excluding caller-supplied inputs, with unit-cost arithmetic.

Two supplied AI label files agreed on all 20 time and space labels. Their identities, prompts, settings, and independence were not verified; they are not expert ground truth. With the optional provider disabled, the analyzer answered all 20 snippets and matched 19 time labels, 20 space labels, and 19 joint labels. All disagreement was in python-S02, reproduced in Listing 1. These descriptive counts locate a diagnostic

discrepancy; they do not validate grading accuracy or generalization.

LISTING 1. ORIGINAL DIAGNOSTIC FIXTURE PYTHON-S02.

```
def solve(a):
    total = 0
    for i in range(len(a)):
        total = 0
        for value in a:
            total += value
    return total
```

At the frozen baseline, the recorded output was O(m*n) time and O(1) space, with heuristic confidence 0.95. Both AI proposals gave $O(n^2)$ and O(1). Here the outer range and inner traversal both use a: for n = len(a), the inner addition executes n^2 times. The matching mathematical specialization is m = n, not an independent dimension. Thus this is a lost size relation and a strict-label mismatch, not evidence that every product bound is incorrect. The reset of total also makes the algorithm functionally wasteful: for nonempty input, only the last recomputed sum is returned.

The issue can arise when separate syntactic traversals, aliases, or helper results refer to the same collection size without carrying that relation into the bound. Conversely, independent arrays of lengths m and n require O(m*n). Replacing every product with $O(n^2)$ would erase genuine multivariate costs, whose importance is established in [8]. The baseline product and quadratic labels map to the same scoring bucket, so this discrepancy alone does not demonstrate a wrong grade. The original output remains in the frozen evidence.

A follow-up repair at commit 752c0d0 resolves direct traversal, range(len(a)), enumerate(a), and simple stable aliases by source identity. It also prevents the substring total += 1 from being mistaken for the pointer update l += 1, which had incorrectly collapsed some nested scans to linear time. Selected unresolved Python cases—opaque iterator bounds or loop-call costs, rebound size aliases, shadowed iteration builtins, and mutation of a traversed collection—now block finalized complexity marks. Confidence, runtime agreement, and model agreement cannot override this structural block; the existing rubric reports an incomplete total without redistributing the missing weights. Thus mitigation includes deterministic identity checks, abstention, and scoring enforcement rather than model fallback alone.

The follow-up passed 359 backend tests, including 23 new regressions, and nine frontend checks, lint, and build. It matched all 20 AI label proposals on rerun. This is a post-hoc repair on a known diagnostic set, not held-out validation. These narrow safeguards do not cover arbitrary aliasing, custom iterators, interprocedural costs, or all languages; heuristic false positives and false negatives remain possible.

Coverage is answered cases divided by eligible cases; exact agreement uses the answered-case denominator, separately for time, space, and joint labels. Both denominators happen to be 20 here. Related cross-language variants and AI proposals preclude an independent-sample accuracy interpretation.

D. Controlled Comparison of Correctness Constraints

To isolate the correctness constraint from weight selection, we hold component vectors and weights fixed across four rules: weighted-only Q, a fixed cap min(Q, 79) when C is below one hundred, multiplicative scaling Q times C divided by one hundred, and min(Q, C). We reuse the twenty-eight-scenario audit grid: seven correctness levels (zero, twenty-five, fifty, seventy-five, ninety, ninety-nine, and one hundred), crossed with four time/space/style triples: 100/100/100, 50/100/80, 100/50/80, and 50/50/80. Three C/T/M/Y profiles (30/35/25/10, 25/25/25/25, and 40/30/20/10) produce eighty-four configurations per rule. This is a synthetic mechanism experiment, not sampled instructor grades.

The violation metric is an unrounded mark greater than C, with a numerical tolerance of one billionth. Table IV shows forty-nine violations for weighted-only aggregation and forty-three for the fixed cap. Both also award nonzero marks in all twelve zero-correctness configurations. Multiplication and the ceiling have no violations. The comparator uses the current weights to isolate the cap's effect; it does not reproduce all historical application settings. The reproducible experiment is frozen at commit 7ddc4aa, while the application snapshot remains d4cdb86.

TABLE IV. CONTROLLED SCORING RULE COMPARISON

Rule	Mark > C (of 84)	Nonzero at C = 0 (of 12)
Weighted only	49	12
Fixed cap 79	43	12
Multiplicative scaling	0	0
Correctness ceiling	0	0

Multiplication also respects the ceiling, because Q and C are each at most one hundred. It is no greater than either operand, and therefore no greater than min(Q, C). The proposed policy selects the largest mark compatible with both the weighted total and the stipulated correctness bound; it avoids another whole-score correctness discount. This is a declared partial-credit preference, not proof that multiplication is educationally inferior.

With default weights and perfect remaining components, increasing C from ninety-nine to one hundred moves the fixed-cap mark from 79 to 100, whereas the ceiling moves from 99 to 100. Separately, at correctness 99.999, ordinary nearest-tenth rounding can display 100 even for min(Q, C); conservative rounding displays 99.9. We separate this presentation check from the unrounded comparison to avoid attributing a rounding difference to the constraint itself.

Weights still matter. Table V summarizes the final unrounded ceiling scores for the same 28 vectors under each profile. The grid is synthetic and equally weighted, so its mean and median describe policy sensitivity, not a student grade distribution. At C = 100, T = M = 50, Y = 80, scores are 68.0, 70.0, and 73.0. None identifies the best rubric without independent instructor judgments.

TABLE V. WEIGHT SENSITIVITY ON THE SAME SYNTHETIC GRID

C/T/M/Y profile	Mean	Median	Q > C
Default 30/35/25/10	55.88	66.35	16/28
Equal 25/25/25/25	56.23	68.62	17/28
Correctness 40/30/20/10	56.92	70.80	16/28

For correctness weight w below one and remaining weighted contribution B, the ceiling is active when C is no larger than B divided by one minus w. Reducing correctness's internal weight can therefore change both grades and the region where optimization becomes visible.

E. Reproducibility and Data Availability

The tagged artifact release [12] provides twenty canonical source fixtures, normalized AI label proposals, provenance hashes, execution reports, frozen-snapshot test transcripts, and reproduction instructions. The tags paper-implementation-20261030 and paper-scoring-experiment-20261030 identify the cited implementation and comparison commits. Proposal aliases are not authenticated reviewer identities; raw review prose is excluded. Published labels retain AI-assisted unverified provenance. The release contains no learner database, credentials, or real private course cases. It supports reproduction of the reported synthetic results, not independent validation of the proposed labels.

VII. Discussion and Limitations

The principal result is consistency between explicit policy and observed implementation behavior. The correctness ceiling prevents efficiency and style from lifting the mark above observed correctness, and the synthetic candidate contract confirms shared scoring in two configurations. Neither result proves that the complexity inference, test suite, or default weights are educationally optimal. Transparency makes a policy inspectable; it does not remove judgment from assessment design.

Several threats constrain interpretation. Test-case correctness depends on specification quality, input serialization, comparator semantics, and coverage of boundary cases. The finite complexity ladder compresses multivariate and library-dependent costs. Confidence is heuristic, and runtime observations can reflect compilation, scheduling, or container overhead. The pilot has two problem groups, related cross-language variants, AI-proposed labels, and no held-out test set. The baseline symbolic-bound failure and separately identified post-hoc repair remain distinguishable; the new tests do not establish robustness on unconstrained student code.

Feedback quality and alignment review remain unvalidated by independent experts. A candidate can pass finite tests yet fail on untested inputs, and a model can approve an unsupported explanation. The guard restricts projection eligibility without guaranteeing a learning outcome. Live-provider success rate and representative latency have not been benchmarked.

Further evaluation can compare complexity estimates with independently adjudicated labels on problem-level holdouts and compare grading policies against blinded instructor judgments. Feedback factuality, relevance, actionability, and disclosure, together with warm/cold latency distributions and provider failures, are separate evaluation targets.

The reported evaluation uses synthetic fixtures and supplied AI review artifacts; it does not analyze identifiable learner records or report participant outcomes. The published package excludes credentials, private course data, and real judge secrets. Imported public problem collections are not redistributed as the publication dataset.

VIII. Conclusion

CodeVedha links execution evidence, resource estimates, style analysis, and optional AI guidance through an auditable assessment contract. Controlled comparison shows that a fixed failure cap does not enforce the chosen correctness bound. Both multiplication and a ceiling enforce it, while the ceiling preserves more partial credit under that declared policy. Execution contracts and candidate-score consistency checks support the implementation, and a diagnostic case study exposes a missing symbolic size relation. Independent expert labels, instructor-grade validation, real-provider evaluation, and learning studies remain necessary for broader accuracy or educational claims.

Acknowledgment

OpenAI Codex was used to draft and revise the abstract and Sections I–VIII, assist with verification scripts and synthetic fixtures, and generate the architecture illustration. The pilot label proposals were supplied as external AI-assisted artifacts, with unverified self-reported model identities. They are reported as AI proposals, not human expert judgments.

References

[1] C. Douce, D. Livingstone, and J. Orwell, “Automatic test-based assessment of programming: A review,” Journal on Educational Resources in Computing, vol. 5, no. 3, Art. no. 4, 2005, doi: 10.1145/1163405.1163409.

[2] H. Keuning, J. Jeuring, and B. Heeren, “A systematic literature review of automated feedback generation for programming exercises,” ACM Transactions on Computing Education, vol. 19, no. 1, Art. no. 3, 2018, doi: 10.1145/3231711.

[3] C. Piech, J. Huang, A. Nguyen, M. Phulsuksombati, M. Sahami, and L. Guibas, “Learning program embeddings to propagate feedback on student code,” in Proceedings of the 32nd International Conference on Machine Learning, vol. 37, 2015, pp. 1093–1102.

[4] V. Sahu, G. R. Gupta, R. Borikar, and N. Mane, “Autograder+: A multi-faceted AI framework for rich pedagogical feedback in programming education,” arXiv:2510.26402, 2025, doi: 10.48550/arXiv.2510.26402.

[5] D. Hendrycks et al., “Measuring coding challenge competence with APPS,” in Proceedings of the NeurIPS Datasets and Benchmarks Track, 2021, arXiv:2105.09938.

[6] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.

[7] S. Gulwani, K. K. Mehra, and T. Chilimbi, “SPEED: Precise and efficient static estimation of program computational complexity,” in Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, 2009. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/popl09_speed.pdf

[8] J. Hoffmann, K. Aehlig, and M. Hofmann, “Multivariate amortized resource analysis,” in Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, 2011, pp. 357–370, doi: 10.1145/1926385.1926427.

[9] A. Jonsson and G. Svingby, “The use of scoring rubrics: Reliability, validity and educational consequences,” Educational Research Review, vol. 2, no. 2, pp. 130–144, 2007, doi: 10.1016/j.edurev.2007.05.002.

[10] Docker, Inc., “docker container run,” Docker Documentation. Accessed: Oct. 30, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/run/

[11] Tree-sitter Contributors, “Tree-sitter: An incremental parsing system for programming tools.” Accessed: Oct. 30, 2026. [Online]. Available: https://github.com/tree-sitter/tree-sitter

[12] CodeVedha Project, “Paper evaluation artifacts and reproduction guide,” CodeAssessment repository, release paper-artifacts-20261030, 2026. Accessed: Oct. 30, 2026. [Online]. Available: https://github.com/multiagentcodereview2026/CodeAssessment/tree/paper-artifacts-20261030/docs/research-paper/artifacts